APOSTILA DE EXERCÍCIOS

TypeScript — prática progressiva

Exercícios de fundamentos, tipagem, funções, objetos, unions, narrowing, generics, utility types e código assíncrono.

Inclui questões com gabarito comentado e três mini projetos de dificuldade crescente, com critérios de conclusão e testes sugeridos.

Objetivo: praticar TypeScript especificamente antes de começar a desenvolver aplicativos.

Como usar esta apostila

Faça os exercícios sem consultar o gabarito. Escreva o código em arquivos .ts e execute a checagem com `npx tsc --noEmit`. Para cada questão prática, tente também criar exemplos de uso e testar casos inválidos.

Pré-requisitos: JavaScript básico (variáveis, condicionais, arrays, objetos e funções) e Node.js instalado. Para praticar, crie uma pasta, execute `npm init -y`, instale TypeScript com `npm install --save-dev typescript` e gere uma configuração com `npx tsc --init`. Ajuste a configuração se necessário.

Regras de estudo

• Prefira strict: true no tsconfig.json.

• Evite any; quando o formato for desconhecido, use unknown e valide.

• Não use as para silenciar erros sem entender a causa.

• Em exercícios com dados externos, lembre-se: tipos TypeScript não validam JSON em tempo de execução.

• O gabarito mostra uma solução possível; soluções equivalentes são válidas se preservarem os requisitos.

Mapa de competências

Bloco	Competências
1–5	Tipos primitivos, inferência e arrays
6–10	Objetos, type/interface, opcionais e readonly
11–15	Unions, narrowing e funções
16–20	Generics, utility types e async
Mini projetos	Modelagem, regras de negócio e testes

Parte 1 — Questões

As questões 1–20 podem ser resolvidas em arquivos separados ou em um único arquivo. Quando a pergunta pedir código, escreva uma implementação funcional e teste-a.

Bloco A — Tipos e inferência

1. Tipos primitivos
Declare variáveis para nome, idade, preço e disponibilidade de um produto. Use string, number e boolean adequadamente.

2. Inferência
Explique quais tipos o TypeScript infere para as declarações abaixo. Depois, diga qual reatribuição seria inválida.

```
let quantidade = 3;
const categoria = "livro";
let habilitado = false;
```

3. Arrays
Crie uma função `media(notas: number[]): number` que devolva a média. Defina o comportamento para uma lista vazia.

4. Tupla
Defina o tipo `Coordenada` como uma tupla de dois números e crie um valor que represente x=12, y=7.

5. Tipos literais
Crie um tipo `Tamanho` que aceite somente "P", "M" ou "G". Escreva uma função que receba esse tipo.

Bloco B — Objetos e contratos

6. Objeto tipado
Defina `Produto` com id numérico, nome textual e preço numérico. Crie dois produtos válidos.

7. Opcional
Adicione a propriedade opcional `descricao` a Produto. Escreva uma função que devolva a descrição ou o texto "Sem descrição".

8. type ou interface
Defina `Usuario` usando interface, com id, nome e email opcional. Depois crie um tipo `Administrador` que acrescente a propriedade nivel: number.

9. readonly
Crie um tipo `Pedido` com id somente para leitura, total e status. Mostre uma atualização válida e explique qual atribuição deve falhar.

10. Utility types
A partir de `Produto`, crie `NovoProduto` sem id e `AtualizacaoProduto` com todas as propriedades opcionais.

Bloco C — Unions, narrowing e funções

11. Union
Crie um tipo `Identificador` que aceite string ou number. Escreva uma função que retorne sempre uma string no formato `ID: valor`.

12. Narrowing
Implemente `formatar(valor: string | number): string`. Para strings, devolva o texto em maiúsculas; para números, devolva duas casas decimais.

13. Union discriminada
Modele o estado de uma requisição com status carregando, sucesso ou erro. O sucesso deve carregar dados string[] e o erro uma mensagem. Escreva uma função que trate os três casos.

14. Parâmetros e retorno
Crie uma função `calcularDesconto(preco: number, percentual: number): number`. Rejeite valores negativos e percentuais acima de 100, usando uma estratégia clara.

15. Callback
Defina o tipo `Transformar` como uma função que recebe string e devolve string. Use-o numa função que transforme uma lista de nomes.

Bloco D — Generics, utilitários e async

16. Generic básico
Implemente `identidade<T>(valor: T): T`. Demonstre que strings e números mantêm seus tipos.

17. Generic com restrição
Crie `tamanhoDe<T extends { length: number }>` que devolva o tamanho de uma string ou array. Por que um número não deve ser aceito?

18. keyof
Crie uma função genérica que receba um objeto e uma chave válida desse objeto, retornando o valor correspondente sem perder seu tipo.

19. unknown
Crie uma função que receba unknown e devolva true apenas se o valor for um objeto não nulo com uma propriedade nome do tipo string.

20. Async
Defina um tipo Usuario e escreva uma função assíncrona que simule buscar um usuário e retorne `Promise<Usuario>`. Trate o caso de ID inválido.

Parte 2 — Mini projetos

Os projetos devem ser feitos sem copiar o gabarito primeiro. Não precisam de interface gráfica, banco de dados ou API real: o objetivo é treinar TypeScript, contratos e lógica. Separe o código em módulos quando fizer sentido e use strict: true.

Mini projeto 1 — Calculadora de preço de venda

Dificuldade: iniciante • Tempo sugerido: 2–4 horas

Objetivo: criar uma calculadora de preço de venda para pequenos produtos. O programa recebe custo, margem desejada e quantidade, valida os dados e produz um resumo.

Tipos obrigatórios: `ProdutoBase`, `ResultadoPreco` e um tipo de erro ou resultado discriminado.

Requisitos funcionais:

• Receber nome do produto, custo unitário, margem percentual e quantidade.

• Validar nome não vazio, custo maior que zero, quantidade inteira positiva e margem entre 0 e 99, inclusive.

• Calcular preço unitário com margem sobre o preço de venda: preço = custo / (1 - margem/100).

• Calcular faturamento estimado e lucro bruto estimado para a quantidade informada.

• Arredondar valores monetários para duas casas na apresentação, sem depender de arredondar cada etapa prematuramente.

• Retornar erros compreensíveis para dados inválidos.

Pratique TypeScript: interfaces/types, funções tipadas, unions, narrowing, validação e separação entre cálculo e formatação.

Exemplo: custo R$ 40, margem 20%, quantidade 5 → preço unitário R$ 50, faturamento R$ 250 e lucro bruto R$ 50.

Critérios de conclusão: os cálculos estão corretos; valores inválidos são rejeitados; nenhuma função principal usa any; há pelo menos 8 testes manuais.

Extensões opcionais

• Adicionar custo fixo por unidade e taxa percentual de venda.

• Criar uma função para comparar margens de 10%, 20% e 30%.

• Exportar o resumo como objeto tipado pronto para ser serializado em JSON.

Mini projeto 2 — Gerador de orçamento e recibo

Dificuldade: intermediário • Tempo sugerido: 4–7 horas

Objetivo: criar a lógica central de uma ferramenta que prepara orçamentos e recibos. Não é necessário gerar PDF de verdade: o resultado pode ser um objeto e um texto formatado.

Tipos obrigatórios:

```
type ItemOrcamento = {
  descricao: string;
  quantidade: number;
  precoUnitario: number;
};

type Documento = {
  id: string;
  cliente: string;
  itens: ItemOrcamento[];
  status: "rascunho" | "enviado" | "aprovado" | "cancelado";
};
```

Requisitos funcionais:

• Adicionar e remover itens de um orçamento.

• Validar descrição, quantidade positiva e preço unitário não negativo.

• Calcular subtotal por item e total do orçamento.

• Aplicar desconto opcional, sem permitir total final negativo.

• Implementar mudanças de status permitidas: rascunho → enviado → aprovado ou cancelado; documentos cancelados não podem ser aprovados.

• Criar um resumo textual para o cliente.

• Criar uma função que transforme um Documento aprovado em um recibo tipado, com data e valor recebido.

Pratique TypeScript: union discriminada, type/interface, readonly quando apropriado, Partial, Pick/Omit, funções puras e tratamento explícito de erros.

Regras de design: não altere o documento original ao adicionar/remover itens; prefira devolver um novo objeto. Separe validação, cálculo, transição de status e formatação.

Critérios de conclusão: pelo menos 12 testes, incluindo transições inválidas, orçamento vazio, desconto excessivo e item inválido.

Extensões opcionais

• Criar tipos distintos para orçamento e recibo, sem confundir documentos ainda não aprovados com documentos pagos.

• Adicionar um identificador genérico reutilizável ou uma função genérica para buscar um documento por id.

• Criar uma função que recebe unknown e valida a estrutura mínima de um documento importado.

Mini projeto 3 — Núcleo de um app de controle financeiro

Dificuldade: intermediário/avançado • Tempo sugerido: 6–10 horas

Objetivo: construir a lógica de um app que registra receitas e despesas, filtra lançamentos e produz um resumo mensal. A prioridade é modelagem rigorosa de tipos, regras e dados de entrada.

Modelo sugerido:

```
type Categoria = "alimentacao" | "transporte" | "moradia" | "salario" | "outros";

type Lancamento = {
  id: string;
  descricao: string;
  valor: number;
  tipo: "receita" | "despesa";
  categoria: Categoria;
  data: string; // ISO: YYYY-MM-DD
};

type ResultadoValidacao<T> =
  | { ok: true; valor: T }
  | { ok: false; erros: string[] };
```

Requisitos funcionais:

• Validar lançamentos importados como unknown, sem confiar apenas em uma asserção de tipo.

• Rejeitar valor não finito, valor menor ou igual a zero, descrição vazia, categoria inválida e data malformada.

• Adicionar, remover e editar lançamentos sem modificar arrays originais.

• Filtrar por mês, tipo e categoria, permitindo combinações de filtros.

• Calcular total de receitas, total de despesas e saldo do período.

• Agrupar despesas por categoria e ordenar lançamentos por data.

• Criar um resumo mensal com tipo de resultado explícito para sucesso ou erro.

• Simular carregamento assíncrono de lançamentos com Promise e tratamento de falha.

Pratique TypeScript: unknown, type guards, unions discriminadas, generics, keyof, utility types, Promise, módulos e strict.

Restrições: não use any; não aceite dados externos sem validação; não misture regras de negócio com formatação; documente as regras de datas e moeda.

Critérios de conclusão: pelo menos 20 testes, incluindo importação malformada, mês sem dados, filtro combinado, saldo negativo e falha assíncrona.

Extensões opcionais

• Criar uma função genérica para ordenar itens por uma chave válida usando keyof.

• Implementar paginação tipada de resultados.

• Definir uma camada de armazenamento em memória por interface e duas implementações diferentes.

• Criar um formato de exportação/importação JSON com validação explícita.

Parte 3 — Plano de testes dos mini projetos

Use esta lista como checklist. Os projetos não estão concluídos apenas porque o caminho feliz funciona.

Projeto	Casos mínimos para testar
1. Preço	Custo zero; margem 0%; margem 99%; margem 100%; quantidade decimal; campos vazios; arredondamento; valores normais.
2. Orçamento	Sem itens; quantidade zero; preço negativo; desconto acima do total; status inválido; aprovação após cancelamento; cliente vazio.
3. Financeiro	JSON não objeto; propriedades ausentes; categoria desconhecida; data inválida; valor Infinity; filtro sem resultados; saldo negativo; Promise rejeitada.

Sugestão de qualidade: escreva testes para os resultados válidos e inválidos. Se ainda não estudou uma biblioteca de testes, comece com chamadas manuais e comparações explícitas; depois experimente Vitest ou outra ferramenta compatível com seu ambiente.

Parte 4 — Gabarito comentado

O gabarito apresenta soluções de referência. Você pode usar outras implementações, desde que respeitem os contratos e os requisitos. Alguns exercícios admitem decisões de projeto diferentes; quando isso acontecer, o essencial é justificar a escolha.

1. Tipos primitivos

```
const nome: string = "Teclado";
const idade: number = 2;
const preco: number = 129.90;
const disponivel: boolean = true;
```

Nome é texto, idade e preço são números e disponibilidade é booleana. Poderia usar const em todos os casos se não houver reatribuição.

2. Inferência

quantidade é number; categoria é o tipo literal "livro" (por ser const); habilitado é boolean. Reatribuir uma string a quantidade ou um valor diferente de "livro" a categoria falharia; habilitado não aceita string.

3. Arrays e média

```
function media(notas: number[]): number {
  if (notas.length === 0) {
    throw new Error("A lista de notas está vazia");
  }
  const soma = notas.reduce((total, nota) => total + nota, 0);
  return soma / notas.length;
}
```

Também seria aceitável devolver NaN ou modelar um resultado de erro. O importante é definir e documentar o comportamento para a lista vazia.

4. Tupla

```
type Coordenada = [number, number];
const ponto: Coordenada = [12, 7];
```

5. Tipos literais

```
type Tamanho = "P" | "M" | "G";

function escolherTamanho(tamanho: Tamanho): string {
  return `Tamanho escolhido: ${tamanho}`;
}
```

6. Objeto tipado

```
type Produto = {
  id: number;
  nome: string;
  preco: number;
};

const p1: Produto = { id: 1, nome: "Caderno", preco: 15 };
const p2: Produto = { id: 2, nome: "Caneta", preco: 3.5 };
```

7. Propriedade opcional

```
type Produto = {
  id: number;
  nome: string;
  preco: number;
  descricao?: string;
};

function obterDescricao(produto: Produto): string {
  return produto.descricao ?? "Sem descrição";
}
```

8. Interface e extensão

```
interface Usuario {
  id: number;
  nome: string;
  email?: string;
}

interface Administrador extends Usuario {
  nivel: number;
}
```

9. readonly

```
type Pedido = {
  readonly id: string;
  total: number;
  status: "aberto" | "pago";
};

const pedido: Pedido = { id: "P-1", total: 50, status: "aberto" };
pedido.total = 60; // válido
pedido.status = "pago"; // válido
// pedido.id = "P-2"; // erro de compilação
```

10. Utility types

```
type Produto = { id: number; nome: string; preco: number };
type NovoProduto = Omit<Produto, "id">;
type AtualizacaoProduto = Partial<Produto>;
```

11. Union

```
type Identificador = string | number;

function formatarId(id: Identificador): string {
  return `ID: ${id}`;
}
```

12. Narrowing

```
function formatar(valor: string | number): string {
  if (typeof valor === "string") {
    return valor.toUpperCase();
  }
  return valor.toFixed(2);
}
```

13. Union discriminada

```
type Estado =
  | { status: "carregando" }
  | { status: "sucesso"; dados: string[] }
  | { status: "erro"; mensagem: string };

function descrever(estado: Estado): string {
  switch (estado.status) {
    case "carregando": return "Carregando";
    case "sucesso": return `Itens: ${estado.dados.length}`;
    case "erro": return `Erro: ${estado.mensagem}`;
  }
}
```

14. Desconto

```
function calcularDesconto(preco: number, percentual: number): number {
  if (!Number.isFinite(preco) || preco < 0) {
    throw new Error("Preço inválido");
  }
  if (!Number.isFinite(percentual) || percentual < 0 || percentual > 100) {
    throw new Error("Percentual inválido");
  }
  return preco * (1 - percentual / 100);
}
```

15. Callback

```
type Transformar = (texto: string) => string;

function transformarNomes(
  nomes: string[],
  transformar: Transformar
): string[] {
  return nomes.map(transformar);
}

const resultado = transformarNomes(["ana", "bia"], nome => nome.toUpperCase());
```

16. Generic básico

```
function identidade<T>(valor: T): T {
  return valor;
}

const texto = identidade("oi"); // string
const numero = identidade(42);  // number
```

17. Generic com restrição

```
function tamanhoDe<T extends { length: number }>(valor: T): number {
  return valor.length;
}

tamanhoDe("TypeScript");
tamanhoDe([1, 2, 3]);
// tamanhoDe(10); // erro: number não possui length
```

18. keyof

```
function obter<T, K extends keyof T>(objeto: T, chave: K): T[K] {
  return objeto[chave];
}

const config = { tema: "escuro", tentativas: 3 };
const tema = obter(config, "tema"); // string
const tentativas = obter(config, "tentativas"); // number
```

19. unknown e validação

```
function temNome(valor: unknown): boolean {
  if (typeof valor !== "object" || valor === null) return false;
  if (!("nome" in valor)) return false;
  return typeof valor.nome === "string";
}
```

Para dados vindos de fora, é comum precisar de validações adicionais para arrays, protótipos ou regras de negócio. Esta solução atende ao requisito mínimo do exercício.

20. Async

```
type Usuario = { id: number; nome: string };

async function buscarUsuario(id: number): Promise<Usuario> {
  if (!Number.isInteger(id) || id <= 0) {
    throw new Error("ID inválido");
  }

  // Simulação de uma fonte assíncrona
  return { id, nome: "Usuário de teste" };
}
```

Uma implementação real deveria consultar a fonte de dados e validar a resposta recebida. A simulação serve para treinar Promise e retorno assíncrono.

Gabarito orientativo — mini projetos

Projeto 1 — Cálculo

```
type ProdutoBase = {
  nome: string;
  custo: number;
  margem: number;
  quantidade: number;
};

type ResultadoPreco =
  | { ok: true; precoUnitario: number; faturamento: number; lucroBruto: number }
  | { ok: false; erros: string[] };

function calcularPreco(p: ProdutoBase): ResultadoPreco {
  const erros: string[] = [];
  if (!p.nome.trim()) erros.push("Nome obrigatório");
  if (!Number.isFinite(p.custo) || p.custo <= 0) erros.push("Custo deve ser positivo");
  if (!Number.isFinite(p.margem) || p.margem < 0 || p.margem >= 100) {
    erros.push("Margem deve estar entre 0 e menos de 100");
  }
  if (!Number.isInteger(p.quantidade) || p.quantidade <= 0) {
    erros.push("Quantidade deve ser um inteiro positivo");
  }

  if (erros.length > 0) return { ok: false, erros };

  const precoUnitario = p.custo / (1 - p.margem / 100);
  const faturamento = precoUnitario * p.quantidade;
  const lucroBruto = (precoUnitario - p.custo) * p.quantidade;

  return { ok: true, precoUnitario, faturamento, lucroBruto };
}
```

A formatação em moeda deve ser separada do cálculo. Use Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL' }) para exibir valores. Em sistemas financeiros de produção, considere as limitações de ponto flutuante e uma estratégia monetária apropriada.

Projeto 2 — Orçamento

Uma solução possível usa uma função pura para cada operação e uma união discriminada para o resultado das transições:

```
type StatusDocumento = "rascunho" | "enviado" | "aprovado" | "cancelado";

type ItemOrcamento = {
  descricao: string;
  quantidade: number;
  precoUnitario: number;
};

type Documento = {
  id: string;
  cliente: string;
  itens: ItemOrcamento[];
  status: StatusDocumento;
};

type Resultado<T> =
  | { ok: true; valor: T }
  | { ok: false; erro: string };

function totalDocumento(doc: Documento): number {
  return doc.itens.reduce(
    (total, item) => total + item.quantidade * item.precoUnitario,
    0
```

```
  );
}

function enviar(doc: Documento): Resultado<Documento> {
  if (doc.status !== "rascunho") {
    return { ok: false, erro: "Somente rascunhos podem ser enviados" };
  }
  return { ok: true, valor: { ...doc, status: "enviado" } };
}

function aprovar(doc: Documento): Resultado<Documento> {
  if (doc.status !== "enviado") {
    return { ok: false, erro: "Somente documentos enviados podem ser aprovados" };
  }
  return { ok: true, valor: { ...doc, status: "aprovado" } };
}
```

Complete com cancelamento, validação de itens, desconto, formatação e recibo. Uma boa solução impede transições inválidas e não altera o documento recebido.

Projeto 3 — Controle financeiro

Uma solução correta deve começar por um validador que recebe unknown e devolve ResultadoValidacao, validar cada campo em tempo de execução e só então permitir que o valor entre na lista tipada. Exemplo de contrato:

```
type ResultadoValidacao<T> =
  | { ok: true; valor: T }
  | { ok: false; erros: string[] };

function validarLancamento(valor: unknown): ResultadoValidacao<Lancamento> {
  // 1. Confirmar que valor é objeto não nulo.
  // 2. Verificar id, descricao, valor, tipo, categoria e data.
  // 3. Retornar { ok: true, valor: lancamentoValidado } se tudo estiver correto.
  // 4. Caso contrário, retornar { ok: false, erros: [...] }.
  throw new Error("Complete a implementação");
}
```

O projeto admite várias soluções. O requisito principal é que nenhum dado externo seja tratado como Lancamento sem validação. Depois, implemente filtros puros, agregações e funções assíncronas com retornos tipados.

Checklist de autoavaliação

- Consigo explicar por que unknown é mais seguro que any.
- Consigo criar tipos de objetos reutilizáveis e propriedades opcionais.
- Consigo modelar estados com unions discriminadas.
- Consigo usar narrowing sem recorrer a as desnecessariamente.
- Consigo escrever uma função genérica simples e explicar T[K].
- Consigo validar unknown antes de usá-lo como dado de domínio.
- Consigo separar validação, regra de negócio e apresentação.
- Consigo executar a checagem do compilador sem erros e interpretar as mensagens.

Se algum item ainda estiver difícil, volte ao bloco correspondente e refaça uma questão sem consultar o gabarito. Você estará pronto para aplicar TypeScript em aplicativos quando conseguir modelar e validar dados, definir contratos claros e testar tanto os caminhos válidos quanto os inválidos.